Observability Interview Prep

Senior Engineer Edition · 14+ YOE · Java / Kubernetes focus

A focused preparation guide for senior-level observability interviews. Covers concepts, architecture, real-world trade-offs, common interview questions, hands-on exercises, and curated resources.

How to use this guide

- **Week 1:** Concepts + Three Pillars + Cardinality + PromQL.
- **Week 2:** OTel architecture, sampling, tracing internals.
- **Week 3:** SLO/SLI/Error budgets, alerting design, on-call.
- **Week 4:** Cost, security, eBPF, AIOps + mock interviews.
- Practice every section's **Self-Check** questions out loud — interviews are verbal.
- Prepare 3 **STAR stories** from your real production work.

Table of Contents

1. Core Concepts & The Three Pillars

Monitoring vs Observability

Monitoring tells you *what* is wrong using predefined dashboards and alerts. It answers **known-unknowns**. **Observability** lets you ask arbitrary questions about your system without shipping new code — it tackles **unknown-unknowns**. The shift is from *thresholds on aggregates* to *high-cardinality, high-dimensional events*.

The Three Pillars

Pillar	Strength	Weakness
Metrics	Cheap, fast aggregation, alerting	Low cardinality, lose context
Logs	Rich detail, free-form context	Expensive at scale, hard to correlate
Traces	End-to-end causality, latency	Sampling loss, infra cost

Senior insight: The three pillars are an industry-popular framing but Honeycomb and others argue the real unit is the **wide structured event**. Be ready to discuss this nuance.

Why Senior Interviews Differ

- Expect **trade-off** questions, not definitions. e.g. "Why pull-based Prometheus over push?"
- Expect **incident war stories** — interviewers want to see how you debug under pressure.
- Expect **architecture whiteboarding** for 50–500 microservice scale.
- Expect **cost** questions: "Your bill grew 4x last quarter. How do you cut it 40% without losing visibility?"

Self-Check

- Define observability without using the word "monitor."
- Give one debugging task where logs win, one where traces win.
- What's an unknown-unknown? Give a concrete example.

2. Metrics Deep Dive — Prometheus & PromQL

Metric Types You Must Explain

Type	Use	Gotcha
Counter	Cumulative count (requests, errors)	Use rate(); resets on restart
Gauge	Snapshot value (CPU%, queue depth)	Aggregation is tricky
Histogram	Latency buckets, computed quantiles	Bucket choice matters; merges OK
Summary	Pre-computed quantiles	Cannot aggregate across instances

PromQL — Patterns Senior Engineers Should Know

Request rate (RPS):

```
rate(http_requests_total[5m])
```

Error ratio (SLI):

```
sum(rate(http_requests_total{status=~"5.."}[5m])) /
sum(rate(http_requests_total[5m]))
```

p99 latency from histogram:

```
histogram_quantile(0.99, sum by (le)
(rate(http_request_duration_seconds_bucket[5m])))
```

Top-K noisy services:

```
topk(5, sum by (service) (rate(errors_total[5m])))
```

Cardinality — The #1 Senior Trap

Each unique *label-value combination* creates a new time series. user_id, request_id, or full URLs as labels can take Prometheus from 1M to 100M series overnight, OOM-killing it. Strategies:

- Use **recording rules** to pre-aggregate high-cardinality data.
- Use **metric_relabel_configs** to drop dangerous labels at scrape.
- Use **VictoriaMetrics, Mimir, or Thanos** when you outgrow single Prometheus.
- Move very high-cardinality signals to **traces or logs** instead.
- Track prometheus_tsdb_head_series as an SLO.

Push vs Pull

Aspect	Pull (Prometheus)	Push (OTLP, StatsD)
Discovery	Server discovers targets	Client must know endpoint
Short-lived jobs	Needs Pushgateway	Native fit
Network	Server reaches into network	Clients reach out (firewall-friendly)
Auth	Centralized at scraper	Distributed across clients

Self-Check

- Difference between rate, irate, increase.
- Why can't you average two summary quantiles across instances?
- Three ways to reduce Prometheus cardinality without losing useful signal.
- When would you choose VictoriaMetrics over Thanos? Over Mimir?

3. Logging Deep Dive

Structured Logging — Non-Negotiable at Senior Level

Plain-text logs are dead at scale. JSON (or logfmt) with consistent fields enables indexing, querying, and correlation. Enforce a **log schema** across services.

```
{"ts":"2026-05-17T10:32:01Z","level":"ERROR","service":"payment","trace_id":"abc123",
"span_id":"def456","user_id":"u-99","event":"charge_failed","reason":"insufficient_fu
nds"}
```

OTel Semantic Conventions

Use OTel semantic conventions for log/metric/trace attribute names (service.name, http.method, http.route, db.system, messaging.system). This is what makes cross-tool correlation work.

Loki vs OpenSearch/Elasticsearch — Trade-offs

Aspect	Loki	OpenSearch / Elasticsearch
Indexing	Labels only (like Prometheus)	Full-text index on all fields
Cost	Cheap (object storage)	Expensive (hot/warm tiers)
Query	LogQL — needs label filter	Powerful DSL, free-text search
Best for	Cloud-native, K8s logs	Security, compliance, deep search

Log-Metric-Trace Correlation

The senior bar: every log line in a request handler must carry **trace_id** and **span_id**. Then Grafana / Honeycomb / Datadog can pivot from a slow trace to its logs in one click. This is achieved by passing the OTel context through the request lifecycle and using log appenders that read it (Logback MDC for Java, structlog for Python).

Self-Check

- Why is structured logging cheaper at query-time than full-text logs?
- How does Loki avoid Elastic-scale storage costs? What's the trade-off?
- Describe trace → log correlation end to end for a Java service.
- What goes in **WARN** vs **ERROR** in your taxonomy?

4. Distributed Tracing

Core Vocabulary

- **Trace:** end-to-end record of a request across services.
- **Span:** single unit of work in a trace (HTTP call, DB query).
- **Context propagation:** passing trace_id between services (HTTP headers, message attributes).
- **W3C TraceContext:** traceparent and tracestate headers — the standard.
- **B3:** older Zipkin format, still common.
- **Baggage:** arbitrary key-values that travel with the trace (e.g. tenant_id).

Sampling Strategies — Critical for Cost

Strategy	When sampled	Pros	Cons
Head-based	At trace start	Cheap, simple, deterministic	Miss rare errors not yet visible
Tail-based	After full trace seen	Keep all errors, slow traces	Needs buffering, expensive infra
Probabilistic	X% randomly	Predictable load	Statistical bias
Adaptive	Variable based on rules	Smart, dynamic	Complex to operate

Senior insight: Tail-based sampling needs a stateful collector layer (OTel Collector with tail_sampling processor) that buffers spans for ~30s — major cost driver and the most-asked design trade-off in tracing interviews.

Context Propagation — Implementation Reality

Auto-instrumentation usually handles HTTP. **Manual** work is required at:

- Async boundaries — thread pools, executors, CompletableFuture.
- Message queues — Kafka headers, RabbitMQ properties.
- Background jobs / scheduled tasks.
- Custom protocols and SDKs without OTel instrumentation.

Self-Check

- Why does head-based 1% sampling lose visibility into rare errors?
- Where do you implement tail-based sampling — in the SDK or Collector? Why?
- How does W3C TraceContext differ from B3?
- Show how you'd propagate a trace across a Kafka producer/consumer in Java.

5. OpenTelemetry Architecture

Collector Anatomy

Every OTel Collector pipeline has three stages:

Stage	Examples
Receivers	otlp, prometheus, jaeger, kafka, filelog
Processors	batch, memory_limiter, attributes, tail_sampling, transform
Exporters	otlp, prometheusremotewrite, loki, opensearch, kafka

Agent vs Gateway Pattern

Agent (DaemonSet on each node): cheap, fast, scrapes local pods, no network hop. **Gateway** (central Deployment): centralizes processing, sampling, auth, tenant routing. **Production standard:** Agent + Gateway. Agents collect; gateways enrich, sample, and route.

Reference config (gateway exporter snippet):

```
exporters: otlp/observability: endpoint: otel-gateway.monitoring:4317 tls: { insecure: false } processors: batch: { send_batch_size: 8192, timeout: 5s } memory_limiter: { limit_mib: 1500, check_interval: 1s } tail_sampling: decision_wait: 30s policies: - { name: errors, type: status_code, status_code: { status_codes: [ERROR] } } - { name: slow, type: latency, latency: { threshold_ms: 1000 } } - { name: rest, type: probabilistic, probabilistic: { sampling_percentage: 5 } }
```

Failure Modes & Scaling

- **Collector OOM** — set memory_limiter processor first in pipeline.
- **Backpressure** — exporters block; use file-storage extension for persistent queue.
- **Single Gateway = SPOF** — run multiple replicas behind a load balancer; use load-balancing exporter for tail-sampling consistency.
- **Slow consumer** downstream — buffer at Gateway with Kafka in between for true decoupling.

Self-Check

- Draw the data flow from a Java app pod to Grafana dashboards in your stack.
- Why is memory_limiter always first in a Collector pipeline?
- How do you achieve tail-based sampling across multiple gateway replicas?
- When would you add Kafka between Collector and backend?

6. SLI / SLO / SLA & Error Budgets

Definitions

Term	Definition	Example
SLI	Quantitative measure of service behavior	Fraction of HTTP 200 in 30 days
SLO	Target value for an SLI	99.9% successful over 30 days
SLA	Contractual SLO with consequences	Refund if <99.5%
Error Budget	1 – SLO over window	0.1% = 43.2 min/month allowed downtime

Writing a Good SLI

Formula: SLI = good events / valid events. Always express as a ratio. Tie it to **user experience**, not internal mechanics.

- **Availability SLI:** successful_requests / total_requests
- **Latency SLI:** requests_under_300ms / total_requests
- **Quality SLI:** requests_with_correct_response / total_requests
- **Freshness SLI:** data_within_5min / total_records

Multi-Window, Multi-Burn-Rate Alerts

Google SRE recommends **two simultaneous burn-rate alerts**:

- **Fast burn:** 2% of monthly budget in 1 hour → page immediately.
- **Slow burn:** 10% in 6 hours → ticket / slack.

PromQL pattern:

```
( sum(rate(errors[1h])) / sum(rate(requests[1h])) ) > (14.4 * 0.001) and (
sum(rate(errors[5m])) / sum(rate(requests[5m])) ) > (14.4 * 0.001)
```

The constants 14.4 and 6 come from the Google SRE workbook tables — burn-rate × SLO error budget. Be ready to derive these.

Self-Check

- Why a 30-day rolling window rather than calendar month for SLOs?
- What's the danger of a 99.99% SLO with two 9's worth of dependencies?
- Explain burn-rate alerting in one minute on a whiteboard.
- What conversation does a depleted error budget force between Dev and SRE?

7. Alerting Design & On-Call

Symptom-Based vs Cause-Based Alerting

Symptom alerts page on user-visible problems (high error ratio, slow p99). **Cause alerts** page on infrastructure (disk full, pod restarts). Senior teams rely heavily on symptom alerts and treat most cause alerts as **warnings/dashboards**.

Reducing Alert Fatigue

- Page only on actionable, urgent, user-impacting alerts.
- Demote noisy alerts to dashboards or slack.
- Group related alerts (Alertmanager group_by).
- Inhibition rules — suppress downstream alerts when upstream firing.
- Track alert volume per on-call shift as a metric; aim to lower it each quarter.

Runbooks & Auto-Remediation

- Every alert MUST link to a runbook URL.
- Runbooks: symptoms, dashboards, queries, fix steps, escalation.
- Use **auto-remediation** for known safe operations (restart unhealthy pod, scale up).
- Track runbook usage and post-incident updates as a team KPI.

On-Call Health

Metric	Healthy range
Pages per on-call shift	< 2
After-hours pages per week	≤ 1
MTTA (acknowledge)	< 5 min
MTTR (resolve)	Depends; track trend
Pager handoff documentation	Always current

Self-Check

- Give an example of a cause alert you would *delete*, and why.
- How would you design alerting for a service with no SLO yet?
- Walk through your last on-call shift. How would you reduce pages 50%?

8. Kubernetes Observability

What to Monitor

Layer	Signals	Source
Cluster	Node CPU/mem, pod restarts, scheduling	node-exporter, kube-state-metrics
Control plane	API server latency, etcd, scheduler	kube-apiserver /metrics
Workload	Pod resource use, OOMKills, throttling	cAdvisor via kubelet
Application	RED metrics, business KPIs	App instrumentation
Service mesh	RPS, latency, error rate per service	Envoy / Istio / Linkerd

PodMonitor vs ServiceMonitor (Prometheus Operator)

- **ServiceMonitor** — selects K8s Services, scrapes their endpoints. Default choice.
- **PodMonitor** — selects Pods directly. Use for Pods without a Service.
- Both create scrape configs dynamically — no manual prometheus.yml edits.

Common Pitfalls

- Logs lost when pod evicted — ship to backend immediately, don't rely on local disk.
- Missing resource limits → monitoring pods OOM the node.
- Ephemeral pod names break dashboards — group by app.kubernetes.io/name labels.
- Default scrape interval 15–30s — too slow for HPA reactions.

Self-Check

- Difference between cAdvisor, node-exporter, kube-state-metrics?
- Why is container_cpu_cfs_throttled_seconds_total often more useful than CPU%?
- How would you observe a slow CrashLoopBackOff?

9. Java / JVM Observability

Critical JVM Metrics

Metric	Why it matters
GC pause time (p99)	Tail latency root cause
GC frequency	Memory pressure indicator
Heap used / max	OOM risk, sizing
Thread count	Leaks, deadlocks
JIT compile time	Cold-start performance
Class loaded count	Memory leaks, classloader issues
Native memory (Metaspace, direct buffers)	Off-heap leaks

Micrometer vs OTel SDK

	Micrometer	OTel Java SDK
Ecosystem	Spring-native, mature in Java	Polyglot, future-proof
Backends	Many vendor exporters	OTLP first, then everything
Tracing	Bridges to OTel since 1.10	Native traces + metrics + logs
Recommendation	Existing Spring stacks	New services, multi-language fleet

Auto-Instrumentation Agent

OTel Java agent is a -javaagent:opentelemetry-javaagent.jar JVM attach. Zero code changes — instruments 100+ libraries (JDBC, Kafka, JDK HttpClient, gRPC, Spring, etc.). Trade-off: ~5–10% overhead, opinionated metric names, harder to debug instrumentation bugs.

Async & Reactive Trace Context

Trace context lives in a ThreadLocal. Crosses thread boundaries fail silently unless:

- Use Context.current().wrap(runnable) for ExecutorService.
- Reactor / Project Reactor: enable Hooks.enableAutomaticContextPropagation().
- Kafka producer/consumer: use OTel Kafka instrumentation or manual TextMapPropagator.

Self-Check

- Walk through a JVM with 99% CPU and rising p99 — what's the diagnostic path?
- When would you NOT use the OTel auto-instrumentation agent?
- How does Logback MDC carry trace_id into structured logs?

10. Incident Response & Debugging Methods

RED Method (for services)

- **Rate** — requests per second.
- **Errors** — failed requests per second.
- **Duration** — latency distribution (p50, p95, p99).

USE Method (for resources) — Brendan Gregg

- **Utilization** — % time resource is busy.
- **Saturation** — queued work waiting on the resource.
- **Errors** — error events on the resource.

Four Golden Signals — Google SRE

- **Latency** — time to service a request.
- **Traffic** — demand on the system.
- **Errors** — rate of failed requests.
- **Saturation** — how full the service is.

Incident Loop

- **Detect** — alert fires.
- **Triage** — is this real? blast radius?
- **Mitigate** — restore service ASAP (rollback, scale, route around).
- **Resolve** — fix the underlying cause.
- **Learn** — blameless postmortem, action items, runbook update.

Postmortem Discipline

Senior engineers are often asked: *"Walk me through a postmortem you led."* Have one ready with: timeline, contributing factors (note: not **root cause** — modern thinking rejects single root causes), what worked, what didn't, action items with owners and dates.

Self-Check

- Which is better for app services: RED or USE? Defend your choice.
- Walk through a recent incident using this loop.
- Why does modern SRE prefer "contributing factors" over "root cause"?

11. Cost, Cardinality & Retention Strategy

The Three Cost Levers

Lever	Tactic
Volume	Drop noisy logs (debug, healthchecks). Tail-sample traces.
Cardinality	Drop high-cardinality labels at collection.
Retention	Hot/warm/cold tiers. Downsample after N days.

Retention Patterns

- Metrics: full resolution 15 days; 5m downsample 90 days; 1h downsample 1 year.
- Logs: hot SSD 7 days; warm 30 days; cold S3 1 year (only for compliance).
- Traces: sample-storage 7 days; keep error/slow traces 30 days.

Cost Senior Interview Question

"Your Datadog bill is $1.2M/year. Reduce by 40% without losing critical visibility. How?"

- Audit top-cardinality metrics — almost always there are 2–3 offenders.
- Drop debug-level logs from production via Collector filter processor.
- Convert chatty info logs into counters / histograms.
- Tail-sample traces with error/slow-path priority.
- Move long-tail retention to a self-hosted Thanos/Mimir bucket.
- Evaluate self-hosted Grafana stack (LGTM) for non-critical workloads.

12. Security & Compliance in Observability

Common Risks

- Logs leaking **PII** (emails, SSNs, card numbers).
- Logs leaking **secrets** (passwords, tokens, API keys).
- Open Prometheus endpoints exposing internal topology.
- Multi-tenant observability with no tenant isolation.

Controls

- Redaction at **collection time** — OTel transform processor with regex.
- Don't log request bodies; allowlist response fields.
- **mTLS** between Collector → Gateway → Backend.
- RBAC on Grafana/Kibana with tenant scoping.
- Encryption at rest for log/trace storage.
- Audit logs for the observability platform itself.

Compliance Frames

Standard	Observability impact
GDPR / CCPA	PII redaction; right to erasure for log data
PCI DSS	No PAN in logs; segregated audit logs
HIPAA	No PHI in logs; access logging
SOC 2	Retention policies, audit trail of access

13. eBPF & Continuous Profiling

Why eBPF Matters for Observability

eBPF runs sandboxed programs in the Linux kernel. For observability, this means zero-code-change instrumentation at syscall, network, and process levels. Examples:

- **Pixie** — auto-instruments K8s apps without code changes.
- **Cilium / Hubble** — service-mesh-grade L3/L7 metrics via eBPF.
- **Parca / Pyroscope / Grafana Profiles** — always-on CPU profiling.
- **Inspektor Gadget** — debugging primitives for K8s.

Continuous Profiling — The Fourth Pillar?

Profiles attribute resource use (CPU, memory, lock contention) to functions. **pprof** format is the open standard. Always-on profiling at 100 Hz adds <1% overhead and answers: *which function is consuming the most CPU right now in production?*

Self-Check

- What's the trade-off between eBPF-based and SDK-based instrumentation?
- When would profiling beat tracing for a latency mystery?

14. AIOps, Anomaly Detection & Modern Trends

Where ML Helps Today

- **Anomaly detection** on metrics — Holt-Winters, Prophet, isolation forest.
- **Log clustering** — Drain3 to group similar log lines, surface novel ones.
- **Alert correlation** — group related alerts into one incident (Big Panda, Moogsoft).
- **RCA assistance** — LLMs summarize incidents from logs/traces (Honeycomb's BubbleUp, Datadog Bits AI).

Where ML Is Mostly Hype

- Fully autonomous remediation — still rare in production.
- Predicting incidents days in advance — accuracy low.
- Replacing on-call engineers — not happening soon.

Modern Stack Trends (2025–2026)

- **OTLP everywhere** — single protocol for metrics, logs, traces, profiles.
- **Vendor-neutral instrumentation** as a hard requirement.
- **Wide events** over three pillars — Honeycomb-style query model gaining ground.
- **Cost-aware observability** — FinOps for telemetry.
- **eBPF** displacing sidecar-based collection.
- **LLM-assisted** incident analysis and dashboard generation.

15. System Design Round — Common Prompts

Design observability for a 100-microservice K8s platform across 3 regions

Cover: instrumentation strategy, collector topology (agent + gateway), backend choice (self-hosted vs vendor), data flow, retention tiers, multi-tenant isolation, cost projection.

Design an SLO program from scratch for a payment service

Cover: SLI definitions, SLO targets per tier, error-budget policy, alerting (burn-rate), monthly review cadence, escalation when budget depleted.

You're getting 1000 alerts/day. Redesign.

Cover: classify by actionability, group, deduplicate, demote, link to SLOs, runbook coverage, metric to track improvement.

Migrate from a single Elasticsearch cluster to a cost-effective log architecture

Cover: traffic analysis, hot/cold tiering, Loki for K8s + OpenSearch for security, cutover plan, data migration, validation.

Add tracing to an existing 50-service Java monolith-to-microservices system

Cover: rollout phasing, auto-instrumentation first, then manual, context propagation across async boundaries, sampling strategy, backend choice.

Build observability for a hybrid edge + cloud system

Cover: edge constraints (bandwidth, intermittent connectivity), buffering, prioritization of telemetry, summarization at edge, cloud reconciliation.

16. Behavioral & STAR Stories

Prepare 3–5 stories using **Situation / Task / Action / Result**. Each should be tellable in 2–3 minutes.

Incident you led

A user-impacting outage where your observability stack helped you find and fix the issue. Quantify MTTR before/after.

Alert redesign

You inherited a noisy alerting system. How did you cut alerts X% while improving real incident detection?

Architectural decision

You chose Prometheus over Datadog (or vice versa). Walk through the criteria, trade-offs, and outcome.

Cost optimization

You reduced observability cost by N% — describe the audit, choices, and what you preserved.

Disagreement

You disagreed with a senior engineer or PM about an SLO target. How did you arrive at consensus?

Mentorship

A junior engineer struggled with an incident. How did you coach them through it?

Senior-level signals interviewers want to hear: ownership without blame, data-driven decisions, explicit trade-off thinking, customer impact framing, follow-through after the incident, cross-team influence.

17. Interview Question Bank (70+)

Concepts

- Define observability without using the word 'monitor.'
- What are unknown-unknowns? Give a real example.
- Why are wide structured events sometimes preferred over three pillars?
- Logs vs metrics vs traces — when does each fail?

Metrics & PromQL

- Counter vs Gauge vs Histogram vs Summary.
- Why use rate() not delta() on counters?
- How does histogram_quantile work? What's its accuracy limit?
- Top 3 ways to control Prometheus cardinality.
- Difference between VictoriaMetrics, Thanos, Mimir, Cortex.
- When would you use remote_write vs federation?

Logging

- Why structured logging at scale?
- Loki vs OpenSearch trade-offs.
- Show how to correlate a slow trace with its logs.
- OTel semantic conventions — name 5 standard attributes.
- How do you prevent PII from leaking into logs?

Tracing

- Head vs tail sampling.
- Why is tail sampling expensive?
- How does W3C TraceContext propagation work?
- Trace context across Kafka — walk through.
- When to use baggage vs span attributes.

OpenTelemetry

- Collector pipeline stages — name each with examples.
- Why is memory_limiter processor first?
- Agent vs Gateway — when do you need both?
- Single OTel Collector deployment is a SPOF — fix it.
- Receivers vs Processors vs Exporters — give 3 of each.

SLO / SLI

- Formula for SLI?
- What's an error budget? Numeric example.
- Why multi-window multi-burn-rate alerts?
- What does it mean when error budget is exhausted?
- How do you avoid over-spec'd SLOs?

Alerting

- Symptom vs cause alerts — give examples of each.
- How do you reduce alert fatigue measurably?
- Alertmanager grouping vs inhibition — when each?
- What's the role of a runbook?
- How would you alert on absence of metrics (dead service)?

Kubernetes

- cAdvisor vs node-exporter vs kube-state-metrics.
- ServiceMonitor vs PodMonitor.
- Diagnose a pod that keeps OOMKilled.
- Why is CPU throttling more useful than CPU%?
- How do you monitor the control plane?

Java / JVM

- List 5 critical JVM metrics.
- Micrometer vs OTel SDK.
- Trace context across CompletableFuture.
- Diagnose a memory leak using observability.
- How does Logback MDC inject trace_id?

Incident & Debugging

- RED vs USE methods.
- Walk through a recent production incident.
- Why prefer 'contributing factors' over 'root cause'?
- How do you run a blameless postmortem?
- Auto-remediation — when is it safe?

Cost & Strategy

- Cut observability bill 40% — how?
- How do you choose between vendor and self-host?
- Retention strategy for logs, metrics, traces.
- What's a cardinality budget?
- When does a service mesh make observability cheaper vs more expensive?

Security

- Common log leak risks.
- How do you enforce PII redaction?
- Securing multi-tenant Grafana.
- Audit trail for the observability platform itself.

Modern Trends

- Where can ML genuinely help observability today?

- What is eBPF and why does it matter for observability?
- Continuous profiling — what problem does it solve?
- Why is OTLP the convergence protocol?

18. Curated Resources

Books (the canonical ones)

- **Site Reliability Engineering** — Google. Free: sre.google/books/
- **The Site Reliability Workbook** — Google. Free online. Hands-on SLO chapter is gold.
- **Observability Engineering** — Majors, Fong-Jones, Miranda (O'Reilly). 2nd edition arriving 2026.
- **Distributed Systems Observability** — Cindy Sridharan (free O'Reilly short book).
- **Systems Performance, 2nd ed.** — Brendan Gregg. USE method origin.
- **BPF Performance Tools** — Brendan Gregg. For eBPF depth.
- **Designing Data-Intensive Applications** — Kleppmann. Not pure obs, but essential context.

Blogs / Newsletters

- **Honeycomb blog** — honeycomb.io/blog (modern observability thinking)
- **Grafana Labs blog** — grafana.com/blog (Mimir, Loki, Tempo, Pyroscope)
- **Charity Majors' blog** — charity.wtf (opinionated, must-read)
- **Cindy Sridharan** — medium.com/@copyconstruct
- **Brendan Gregg** — brendangregg.com (perf + eBPF)
- **Liz Fong-Jones** — talks & posts on SLOs and on-call health
- **SRE Weekly** — newsletter (sreweekly.com)
- **DZone SRE / Observability zone** — community articles, mixed quality but good breadth

Free Courses / Tutorials

- **Coursera — Google SRE: Measuring & Managing Reliability** (free to audit)
- **KodeKloud — Prometheus Certified Associate (PCA) prep**
- **Linux Foundation — Intro to OpenTelemetry (LFS148)** (free)
- **Grafana Labs Webinars + Tutorials** — grafana.com/tutorials
- **CNCF YouTube** — KubeCon observability talks (search: "observability KubeCon")
- **O'Reilly Learning Platform** — Observability Engineering video course

Talks Worth 30 Minutes

- Charity Majors — "Observability: A Manifesto" (multiple conferences)
- Liz Fong-Jones — "Adopting SLOs" (SRECon)
- Ben Sigelman — "Three Pillars, Zero Answers" (KubeCon)
- Brendan Gregg — "BPF Performance Tools" (LISA)
- Cindy Sridharan — "Monitoring in the Time of Cloud Native"

Hands-on Labs

- Spin up **Prometheus + Grafana + OTel Collector + Tempo + Loki** via the Grafana LGTM stack docker-compose.

- Instrument a Java Spring Boot app with the **OTel Java auto-instrumentation agent**.
- Build a Prometheus dashboard from scratch (RED + USE) for one of your real services.
- Write 5 PromQL queries: rate, error ratio, p99, top-K, predict_linear for disk full.
- Implement **tail-based sampling** in OTel Collector and verify trace selection.
- Define an SLO for one service and write the burn-rate alert PromQL.

Reference Docs (bookmark these)

- OpenTelemetry: opentelemetry.io/docs/
- Prometheus PromQL: prometheus.io/docs/prometheus/latest/querying/basics/
- Grafana Loki LogQL: grafana.com/docs/loki/latest/query/
- Google SRE books (free): sre.google/books/
- OTel Semantic Conventions: opentelemetry.io/docs/specs/semconv/

19. 4-Week Study Plan

Week 1 — Foundations

- Re-read Three Pillars + observability vs monitoring framing.
- PromQL: rate, irate, histogram_quantile, recording rules — 20 queries on real data.
- Cardinality theory + 3 real reduction case studies.
- Pillar deep-dives: metrics types, structured logging, trace anatomy.
- Do all Self-Check questions in Sections 1–3 out loud.

Week 2 — Architecture & Tracing

- OTel Collector pipelines: build one locally with multiple receivers/exporters.
- Agent + Gateway hands-on with K8s.
- Tail vs head sampling: implement both, observe data loss differences.
- Trace context across Kafka and async — code it.
- Mock interview yourself on: "Design observability for 100 services."

Week 3 — SLO, Alerting, K8s, Java

- Write an SLO end-to-end for one of your services with burn-rate alerts.
- Audit alerts in your current job — categorize symptom vs cause.
- K8s observability: PodMonitor, kube-state-metrics, control-plane signals.
- JVM deep-dive: pause time, throttling, classloader, native memory.
- Prepare 3 STAR stories.

Week 4 — Modern, Cost, Mock Loop

- Cost / cardinality optimization scenarios.
- Security: PII redaction in OTel Collector.
- eBPF + continuous profiling: read one Pixie blog, one Parca blog.
- AIOps survey: where it works, where it doesn't.
- Do 2–3 full mock interviews — concepts, system design, behavioral.

Daily habit: 15 min reading + 30 min hands-on + 15 min self-check questions out loud. Speaking the answers is what distinguishes someone who *knows* from someone who can *perform* in an interview.